

Arete - Technical Documentation

STA561D: Probabilistic Machine Learning | Duke University | Spring 2026

1. How I Built This With Claude

This section is an honest account of the division of labour between me and Claude in building Arete. It is written in the spirit of the course guidelines, which explicitly encouraged the use of AI to accelerate coding, synthesis, and development, while retaining the intellectual contribution and creative direction.

1.1 What I Contributed

The central idea of Arete is mine: that the pedagogically best move is not the objectively best move - The same question runs through my broader research interest: when an algorithm knows the optimal decision, what determines how much of that value a human decision-maker realises. The statistical arm of that question - determining the upper bound of what an adaptive recommendation system can deliver under uncertainty, and how close any realised policy comes to that ceiling - is the thread that connects this project to reinforcement learning for retail allocation. In this case, the chess tutor is designed to optimise for teaching rather than winning. Every architectural decision followed from this framing.

I identified Stockfish Skill Level as the right mechanism for ELO-calibrated move generation, recognising it as behavioural cloning rather than true RL and being explicit about that distinction from the start. I specified the Bayesian ELO estimator as the right inferential framework for latent skill estimation, not a simpler heuristic. I designed each of the seven experiments, including the ELO recovery test as a stricter validity check than readability metrics alone, the temperature hypothesis to give the skill parameter a probabilistic grounding, and the strategy-ELO interaction test to probe whether style injection compounds with ELO calibration.

I decided to integrate the RL agent into the Streamlit app as a playable mode rather than leaving it as a notebook demonstration. I named the system Arete and identified the central inversion that the name carries. Every document went through multiple rounds of review and directed revision. Every experiment result was read, interpreted, and either validated or challenged before being included in the submission.

1.2 What Claude Contributed

Claude wrote all the code. Every Python file, every notebook, every Streamlit component was written by Claude to my architectural specification. When code broke, I provided the error output and context; Claude diagnosed and fixed it. When I wanted a new feature, I

described what it should do and Claude implemented it. The mathematical notation in the technical documentation was written by Claude. The first drafts of all three submission documents were written by Claude based on my rather unstructured inputs and revised through directed feedback.

Claude also suggested experiments I had not considered, some of which I adopted, others I rejected. The depth-variance uncertainty quantification and the TF-IDF cosine similarity approach for the strategy-ELO interaction were Claude's suggestions that I validated and included. The specific prompt language for the four engineering strategies was Claude's implementation of my requirement to compare approaches systematically.

1.3 The Joint Work

Debugging was joint: I ran the code and provided the error messages and screenshots and the context on where it is failing, Claude diagnosed the cause and proposed the fix, I tested the fix and confirmed or escalated. The Windows asyncio issue, the Stockfish path resolution, the RL agent starting position redesign across three iterations, and the endgame board rendering were all debugged this way.

Document iteration was joint: I started off with my idea document and Claude gave me a first draft. I reviewed each draft, identified what was wrong in terms of voice, framing, or accuracy, and directed the revision. Claude rewrote to the specification. The executive summary went through seven versions before reaching the final form. This process is analogous to working with a skilled research assistant who can write and code but whose judgment about what matters and what does not belong to the principal investigator.

1.4 Assessment

The experimental design, research framing, and intellectual direction are mine. The implementation velocity is attributed to Claude. A project of this scope, seven experimental notebooks, a multi-mode Streamlit application with five probabilistic features, a trained RL agent integrated as a playable mode, and three submission documents, would not have been completable without Claude handling the coding and drafting. The brain was not outsourced. The hands were.

2. System Architecture

Arete is composed of four Python modules and a Streamlit front-end, deployed on a local laptop without server infrastructure.

2.1 engine.py — Stockfish Interface

- Wraps python-chess for board state management, legal move generation, and FEN parsing
- Interfaces with Stockfish 18 via `chess.engine.SimpleEngine.popen_uci()`
- ELO to Skill Level mapping: 800 to 1, 1000 to 3, 1200 to 5, 1400 to 8, 1600 to 11, 1800 to 14, 2000 to 17, 2200 to 19, 2400 to 20
- `get_position_complexity()`: samples Stockfish 15 times, computes Shannon entropy, classifies as Clear, Moderate, or Complex
- `get_evaluation_confidence()`: evaluates at depth 5 and depth 15, computes gap, classifies as High, Medium, or Low
- `get_position_evaluation_numeric()`: returns centipawn score as float capped at plus or minus 10 for chart rendering
- Windows asyncio fix: `WindowsProactorEventLoopPolicy` required for python-chess subprocess transport

2.2 tutor.py — LLM Explanation Layer and Bayesian Estimator

- Calls Anthropic API (claude-haiku-4-5) with structured prompts
- Six ELO profiles each specifying: vocabulary rule with forbidden term list, explanation depth, planning horizon, analogy guidance
- Four strategy profiles (Aggressive, Solid, Positional, Balanced) injected as `move_bias` and `plan_bias`
- Three mandatory output sections enforced: Why this move, Your strategy going forward, How does this compare
- `get_alternative_moves()` pulls two actual legal moves from the board for the comparison section
- `BayesianELOEstimator` class: maintains posterior over nine ELO bands, Gaussian likelihood model, MAP estimate drives displayed ELO, posterior std tracked as uncertainty measure

2.3 endgame_env.py — RL Environment

- 5x5 King+Pawn vs King environment with randomised starting positions
- White wins by promoting pawn to row 5 (reward +1.0), Black wins by capturing pawn (reward -1.0)
- Step penalty of -0.02 per White move and row advancement reward of +0.3 per pawn row advanced for reward shaping
- `render_svg()` method produces base64-encoded SVG board for Streamlit rendering
- `black_heuristic()` implements capture-first then approach policy for the opponent

2.4 app.py — Streamlit UI

- Three modes: Tutor Mode, Play vs Bot, Endgame Trainer
- Bayesian ELO panel: sidebar toggle, posterior bar chart via Altair, feedback buttons (Too Simple, Right Level, Too Complex), MAP estimate drives displayed ELO
- Position complexity and confidence computed on each analysis call, displayed below board
- Position trend chart: Altair area chart with green/red coloring updated after each move
- Endgame Trainer: loads pickled Q-table via st.cache_resource, renders 5x5 board, player selects White moves, agent responds, coaching tips via direct Anthropic API call
- Move input: From/To square selector dropdowns, auto-handles pawn promotion to queen

3. Mathematical Formulations

3.1 Temperature Model for Move Generation

Stockfish at Skill Level k samples moves from a Boltzmann distribution:

$$P(m | k) \propto \exp \left(\frac{Q(m)}{\tau(k)} \right)$$
$$P(m | k) \propto \exp \left(\frac{Q(m)}{\tau(k)} \right)$$

where $Q(m)$ is the centipawn advantage of move m and $\tau(k)$ is an implicit temperature. Low k yields high τ and near-uniform distribution over moves. High k yields low τ and concentration on $\operatorname{argmax} Q(m)$. Shannon entropy of the empirical distribution:

$$H(k) = - \sum_m \hat{p}(m | k) \log_2 \hat{p}(m | k)$$

Spearman rank correlation between Skill Level and H : $\rho = -0.905$, $p = 0.0020$.

3.2 Bayesian ELO Estimation

Let e^* be the latent true ELO, e_{disp} the displayed ELO, f the feedback signal. Prior: uniform over nine ELO bands. Likelihood with $\delta = e^* - e_{\text{disp}}$ and threshold $\theta = \frac{\sigma}{2}$:

$$P(\text{Too Simple} \mid \delta) = 1 - \Phi\left(\frac{\theta - \delta}{\sigma}\right)$$

$$P(\text{Right Level} \mid \delta) = \Phi\left(\frac{\theta - \delta}{\sigma}\right) - \Phi\left(\frac{-\theta - \delta}{\sigma}\right)$$

$$P(\text{Too Complex} \mid \delta) = \Phi\left(\frac{-\theta - \delta}{\sigma}\right)$$

Posterior update via Bayes rule:

$$P(e^* \mid f_1, \dots, f_n) \propto \prod_i P(f_i \mid e^*, e_{\text{disp},i}) \cdot P(e^*)$$

MAP estimate: $\hat{e} = \arg \max_e P(e \mid f_1, \dots, f_n)$

$$\text{Posterior std: } \sigma_{\text{post}} = \sqrt{\sum_e P(e \mid \text{data}) \cdot (e - \mu_{\text{post}})^2}$$

Optimal sigma = 300 ELO points from sensitivity analysis.

3.3 Position Complexity

$$H = - \sum_m \hat{p}(m) \log_2 \hat{p}(m), \quad \hat{p}(m) = \frac{\text{count}(m)}{n_{\text{samples}}}$$

Thresholds: H less than 0.8 bits is Clear, 0.8 to 1.8 bits is Moderate, above 1.8 bits is Complex.

3.4 Evaluation Confidence

$$\Delta = |v_{15} - v_5| \text{ (pawn units)}$$

Thresholds: Delta less than 0.3 is High, 0.3 to 0.8 is Medium, above 0.8 is Low.

3.5 Monte Carlo Control Q-Update

$$Q(s, a) \leftarrow Q(s, a) + \frac{1}{N(s, a)} (G - Q(s, a))$$

$$\varepsilon(s) = \frac{N_0}{N_0 + N(s)}, \quad N_0 = 100$$

First-visit MC with gamma = 1 (no discounting). Reward shaping: +0.3 per pawn row advanced, -0.02 step penalty.

4. Experimental Results

4.1 Temperature Analysis

- Open Middlegame: Spearman rho = -0.905, $p = 0.0020$, entropy range 2.22 to 0.99 bits
- Constrained Endgame: entropy = 0 across all skill levels, one move dominates regardless of skill parameter
- Implied softmax temperature decreases monotonically with Skill Level

4.2 Readability Baseline

- ELO 800: FK Grade 6.8, Flesch Ease 73.5, Syllables/Word 1.39, Advanced Terms 0
- ELO 1200: FK Grade 7.7, Flesch Ease 61.4, Syllables/Word 1.57, Advanced Terms 1
- ELO 1600: FK Grade 10.3, Flesch Ease 41.8, Syllables/Word 1.81, Advanced Terms 2
- ELO 1800: FK Grade 11.4, Flesch Ease 32.9, Syllables/Word 1.92, Advanced Terms 5

4.3 Prompt Strategy Comparison

- Monotonicity Score: all four strategies tied at 0.67
- FK Range (ELO 800 to 1800): Structured 2.79, Minimal 2.46, Chain-of-thought 2.28, Role-play 1.95
- Avg Advanced Terms at Low ELO: Role-play 0.0, Chain-of-thought 0.0, Minimal 0.5, Structured 0.5

4.4 ELO Recovery

- Classification accuracy: 25 percent (1 of 4 correct)
- Mean ELO error on misclassifications: 250 ELO points
- Classifier predicted ELO 1600 for three of four inputs with high confidence
- Finding: surface readability calibrates cleanly, semantic depth does not

4.5 Bayesian ELO Estimation

- 100 percent of simulated players identified within one ELO band after 20 rounds
- Exact match rates: ELO 800 at 94 percent, ELO 1200 at 82 percent, ELO 1400 at 76 percent, ELO 1800 at 88 percent, ELO 2200 at 91 percent
- Posterior std reduces from 516 to median 104 ELO points: 80 percent uncertainty reduction
- Optimal sigma: 300 ELO points from sensitivity analysis

4.6 Strategy-ELO Interaction

- Cosine similarity range: 0.129 to 0.179 across ELO bands
- Peak differentiation at ELO 1600, not ELO 1800 as hypothesised
- Hypothesis not confirmed: strategy differentiation does not increase monotonically with ELO

4.7 Positional Complexity and Faithfulness

- Opening faithfulness: 0.834 (best)
- Middlegame faithfulness: 0.678
- Endgame faithfulness: 0.688
- ELO 1800 Middlegame: 0.43, worst single result, strategic abstraction overtakes positional specificity

4.8 Depth-Variance Uncertainty

- Quiet Opening: low std across depths, evaluation converges early
- Tactical Middlegame: moderate depth-sensitivity
- Sharp Tactical: highest variance, evaluation shifts significantly with depth

4.9 Reinforcement Learning Endgame

- Episodes: 500,000 with randomised starting positions
- States visited: 2,138 vs 20 with fixed starting position
- Q-values: 12,680 non-zero state-action pairs
- Win rate progression: 86.2 percent at 100k, 93.2 at 200k, 95.9 at 300k, 96.5 at 400k, 97.6 at 500k
- Bulk evaluation on 1,000 random positions: 100 percent White wins with greedy policy

- All four held-out test positions solved correctly including rook pawn and distant king coordination cases

5. Reproduction Instructions

Repository: <https://github.com/PinakiG-duke/sta561-chess-tutor>

Environment setup:

```
conda create -n chess_tutor python=3.11 -y
conda activate chess_tutor
pip install -r requirements.txt
```

.env configuration (copy from .env.example):

```
ANTHROPIC_API_KEY=your_key_here
STOCKFISH_PATH=path/to/stockfish.exe
```

Run application:

```
streamlit run app.py
```

Run any notebook:

```
jupyter notebook notebooks/<notebook_name>.ipynb
```

6. Change Log

1. **Initial build:** Three-file architecture, Stockfish move generation, Anthropic explanation layer, Streamlit UI, two modes (Tutor and Play vs Bot), UCI dropdown move input, .env configuration.
2. **Windows asyncio fix:** WindowsProactorEventLoopPolicy added to engine.py.
3. **Stockfish path resolution:** Forward slashes in .env for cross-platform compatibility.
4. **ELO-calibrated explanations:** Explicit forbidden vocabulary lists, three mandatory output sections, comparison against actual legal moves, strategy depth scales with ELO.
5. **Position evaluation chart:** Altair area chart with green and red coloring rendered after each move.

6. **Strategy selector:** Four styles injected into explanation and commentary prompts.
 7. **Move input improvement:** From/To square selector replaces raw UCI dropdown, auto-handles pawn promotion.
 8. **Position Complexity Meter:** Shannon entropy of Stockfish move distribution, 15 samples, Clear/Moderate/Complex classification.
 9. **Evaluation Confidence Indicator:** Depth-5 vs depth-15 gap, High/Medium/Low classification with colour indicator.
 10. **Bayesian ELO Estimator:** Full Bayesian model in tutor.py, sidebar toggle, posterior bar chart, feedback buttons, MAP estimate drives displayed ELO.
 11. **Endgame Trainer mode:** KingPawnEnv in endgame_env.py, MC Control agent trained and pickled, third app mode with 5x5 board rendering, coaching tips via direct API call.
 12. **System renamed to Arete:** App title and caption updated, documentation updated throughout.
-

7. Known Limitations and Future Work

13. **Semantic calibration gap:** ELO recovery accuracy 25 percent. Primary fix: Maia Chess for ELO-appropriate move content.
14. **No dominant prompt strategy:** Hybrid combining Structured vocabulary range and Role-play compliance is recommended next step.
15. **High-ELO faithfulness degradation:** ELO 1800 middlegame faithfulness 0.43. Prompt redesign needed to maintain positional grounding.
16. **Strategy-ELO interaction null result:** Style differentiation does not increase with ELO. More integrated solution: bias Stockfish move selection by style.
17. **RL northstar:** Full RL-based move generation requires neural network Q-function, TD-learning, and MCTS. The endgame trainer is the proof of concept. Maia Chess is the near-term step.
18. **Click-to-move:** Current move input uses square selectors. Click-to-move requires JavaScript chess component as Streamlit custom component.
19. **No persistent memory:** System does not track mistake patterns across sessions. Persistent player profiles are a planned extension.